

# Ускорение инференса UNet-модели

Бельский Антон

Курилов Олег

Комогорцев Андрей

Курс “Эффективные модели ML и архитектуры нейросетей”

Апрель 2026

## 1 Введение и постановка задачи

### 1.1 Цель проекта

Целью данного проекта является исследование и сравнение методов ускорения инференса свёрточной нейронной сети с архитектурой UNet. В качестве базового подхода используется стандартный инференс в половинной точности (FP16) на графическом процессоре. В дальнейшем планируется применить `torch.compile`, альтернативный компилятор TVM, а также методы квантизации и спарсификации (pruning, 2:4 semi-structured sparsity). Основными метриками сравнения являются задержка (latency) в миллисекундах на одно изображение, пропускная способность (throughput) и качество сегментации (Dice коэффициент). Итоговым результатом должна стать воспроизводимая экспериментальная платформа и сравнительная таблица, демонстрирующая вклад каждого метода.

### 1.2 Выбранные данные

Для экспериментов выбран датасет **Carvana Image Masking Challenge** [1]. Датасет содержит изображения автомобилей и соответствующие маски сегментации. Размер изображений варьируется, однако для унификации и ускорения загрузки все изображения и маски были приведены к разрешению, уменьшенному в 2 раза (коэффициент масштабирования 0.5) с сохранением пропорций. В качестве источника данных использовались файлы, размещённые на Google Drive. Поскольку официальные тестовые маски отсутствовали в открытом доступе, все замеры производились на обучающей выборке. Это не влияет на корректность сравнения, так как модель заморожена (веса не изменяются), а нас интересуют исключительно временные характеристики инференса. Для воспроизводимости выбрано фиксированное количество изображений (1280), а порядок батчей сделан детерминированным (без перемешивания).

### 1.3 Архитектура модели

Используется предобученная модель UNet [2] из репозитория milesial/Pytorch-UNet [3]. Модель обучена на датасете Carvana для задачи бинарной сегментации (фон /

автомобиль). Архитектура представляет собой классическую UNet с энкодером, состоящим из свёрточных блоков, и декодером с апконволюциями. Загрузка весов осуществлена через `torch.hub`. Все эксперименты проводятся в режиме оценки (`eval()`) с отключённым вычислением градиентов.

## 1.4 Программное и аппаратное обеспечение

- **Графический процессор:** NVIDIA Tesla T4 (16 ГБ видеопамати).
- **Операционная система:** Google Colaboratory (Linux).
- **Версии ключевых библиотек:**
  - Python 3.12
  - PyTorch 2.10.0+cu128
  - CUDA 12.8
  - torchmetrics 1.9.0
  - NumPy 2.0.2
  - PIL (Pillow)

## 1.5 Особенности методики бенчмаркинга

Для получения честных замеров времени инференса использовались следующие правила:

- Перед каждым измерением выполнялась синхронизация GPU (`torch.cuda.synchronize()`).
- Измерялось только время выполнения модели (`model(images)`), без учёта загрузки данных и предобработки.
- Для каждого конфига проводился прогревочный батч, результаты которого не учитывались.
- Количество обрабатываемых изображений фиксировано (1280), количество батчей вычислялось как  $\lceil 1280 / \text{batch\_size} \rceil$ .
- В случае использования смешанной точности (FP16) применялся `torch.autocast(device_type="cuda", dtype=torch.float16)` и модель переводилась в `.half()`.
- Качество сегментации оценивалось с помощью метрики Dice (BinaryF1Score) на всех обработанных изображениях, чтобы убедиться в отсутствии деградации после оптимизаций.

## 2 Результаты первого эксперимента и их анализ

### 2.1 Сравнение FP16 baseline и torch.compile

В первом эксперименте проведено сравнение двух подходов к инференсу UNet на датасете Carvana:

- **FP16 baseline** — модель в половинной точности, стандартный цикл инференса без дополнительных оптимизаций;
- **torch.compile** — модель компилируется с помощью `torch.compile` (режим по умолчанию), после чего инференс выполняется в FP16.

Эксперимент выполнен для размеров батча 2, 4, 8, 16. Количество обработанных изображений фиксировано (1280), время замерялось только на этапе прямого прохода модели, без учёта загрузки данных. Для каждого конфига проводился прогревочный батч, результаты которого исключались из статистики. Качество сегментации оценивалось с помощью метрики Dice (BinaryF1Score) на всех обработанных изображениях.

### 2.1.1 Сводные данные

Таблица 1: Сравнение времени инференса и качества

| Batch size | Метод         | Время на батч, с | Время на изобр., мс | Dice    |
|------------|---------------|------------------|---------------------|---------|
| 2          | baseline fp16 | 0.1588           | 79.38               | 0.99125 |
| 2          | torch.compile | 0.1294           | 64.69               | 0.99125 |
| 4          | baseline fp16 | 0.3148           | 78.70               | 0.99125 |
| 4          | torch.compile | 0.2646           | 66.14               | 0.99125 |
| 8          | baseline fp16 | 0.6384           | 79.80               | 0.99125 |
| 8          | torch.compile | 0.5398           | 67.48               | 0.99125 |
| 16         | baseline fp16 | 2.0696           | 129.35              | 0.99125 |
| 16         | torch.compile | 1.8702           | 116.89              | 0.99125 |

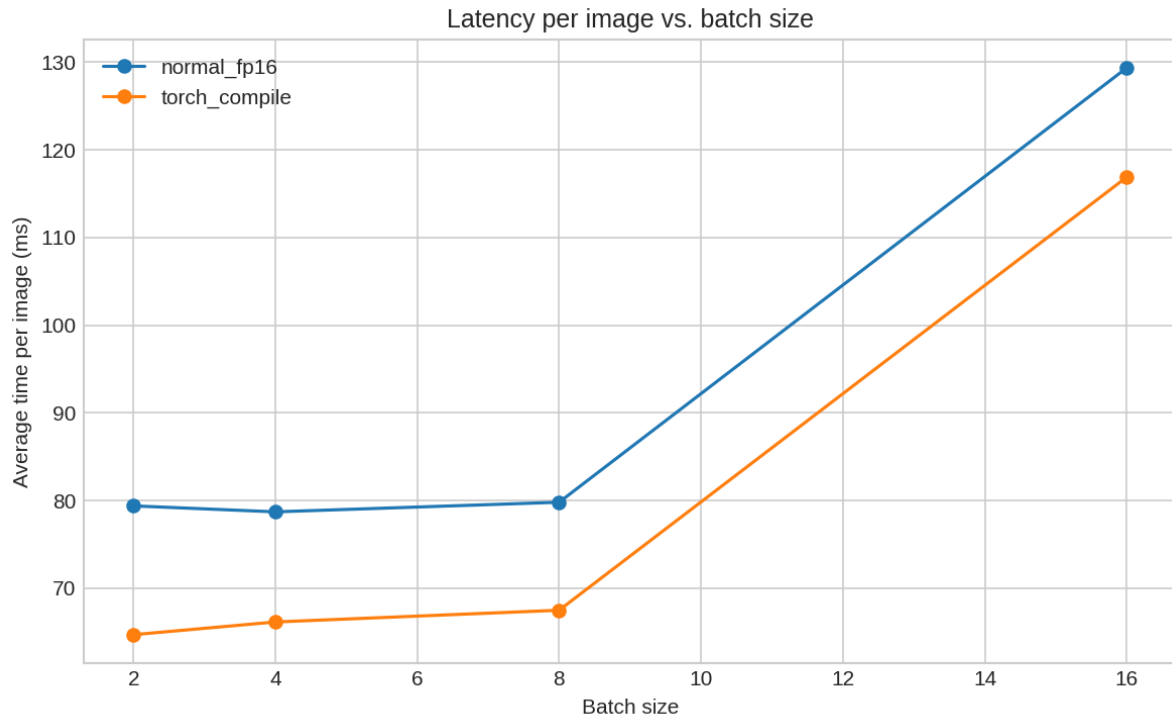


Рис. 1: Среднее время инференса на одно изображение в зависимости от размера батча.

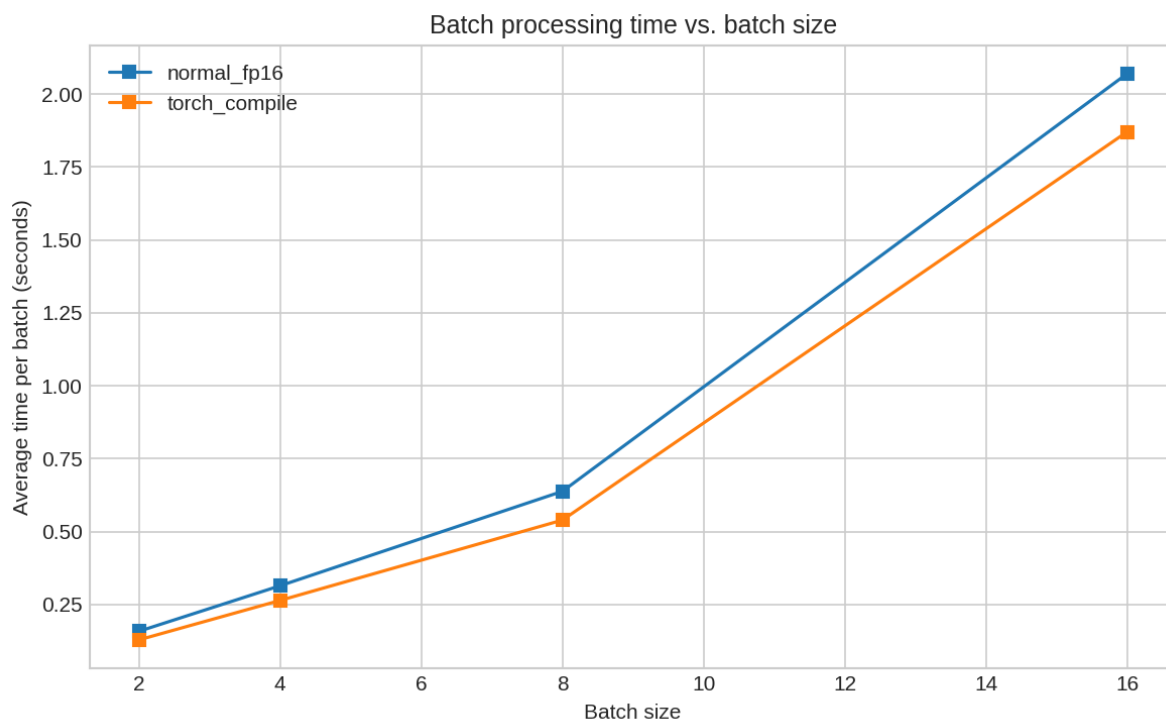


Рис. 2: Среднее время обработки одного батча в зависимости от его размера.

## 2.2 Анализ полученных результатов

### 2.2.1 Влияние `torch.compile`

`torch.compile` выполняет графовую оптимизацию вычислений: он анализирует граф операций, выполняет слияние ядер (kernel fusion), устраняет избыточные операции и генерирует более эффективный код для GPU. В результате сокращается количество запусков ядер и уменьшаются накладные расходы на пересылку данных между глобальной и разделяемой памятью (global memory и shared memory).

Из таблицы и графиков видно, что `torch.compile` даёт устойчивое ускорение во всех конфигурациях:

- Для batch size 2–8 ускорение составляет 15–18%.
- Для batch size 16 ускорение несколько ниже — около 10%.
- Качество (Dice) остаётся практически неизменным, что подтверждает корректность компиляции.

### 2.2.2 Аномальное поведение при batch size 16

Обращает на себя внимание резкое увеличение времени на изображение при переходе от batch size 8 к 16: для baseline с 79.8 мс до 129.4 мс, для compile с 67.5 мс до 116.9 мс. В то же время для batch size 2–8 время на изображение остаётся практически постоянным. Такое поведение свидетельствует о смене доминирующего фактора, ограничивающего производительность.

Оценим характеристики GPU NVIDIA T4, на котором проводились эксперименты:

- Объём кэша L2: 4 МБ.

- Пропускная способность памяти: 320 ГБ/с.
- Пиковая производительность FP16 (тензорные ядра): 65 TFLOPS (теоретическая).

При малых батчах (2, 4, 8) общий объём активаций и промежуточных данных, необходимых для вычислений, уместается в кэш L2. Это позволяет избежать постоянных обращений к глобальной памяти, и время определяется главным образом накладными расходами на запуск ядер и их выполнением. При batch size 16 объём данных становится слишком большим для кэша L2 (например, только промежуточные активации могут занимать несколько мегабайт), и GPU вынужден постоянно подгружать данные из глобальной памяти. Это приводит к увеличению времени доступа и, как следствие, к скачку latency.

Кроме того, при увеличении размера батча растёт конкуренция за ресурсы внутри мультипроцессоров (SM): больше warp-ов одновременно находятся в работе, что может приводить к конфликтам при доступе к разделяемой памяти и к дополнительным задержкам.

### 2.2.3 Оценка compute и memory нагрузок

Для количественного анализа оценим объём операций и данных для одного изображения. Модель UNet с энкодером ResNet34 имеет около 28 млн параметров (в FP16 — 56 МБ). Входное изображение после масштабирования 0.5 имеет размер около  $600 \times 400$ , для упрощения примем  $512 \times 512$  (1.5 МБ). Полное число операций (FLOPs) для такого разрешения оценивается в 10–15 GFLOP на изображение. Возьмём  $F_{\text{img}} = 12$  GFLOP. Объём данных, пересылаемых за один проход (веса, входы, выходы, активации), сложно оценить точно, но для одного изображения он может составлять сотни мегабайт. Примем  $D_{\text{img}} \approx 300$  МБ.

Тогда для batch size 8:

$$\begin{aligned} F_{\text{total}} &= 8 \times 12 \text{ GFLOP} = 96 \text{ GFLOP}, \\ D_{\text{total}} &= 8 \times 300 \text{ МБ} = 2.4 \text{ ГБ}. \end{aligned}$$

Арифметическая плотность:

$$\frac{F_{\text{total}}}{D_{\text{total}}} \approx \frac{96 \text{ GFLOP}}{2.4 \text{ ГБ}} = 40 \text{ FLOP/байт}.$$

Теоретическая плотность, которую может обеспечить GPU:

$$\frac{P_{\text{peak}}}{B_{\text{peak}}} \approx \frac{65 \times 10^{12} \text{ FLOP/с}}{320 \times 10^9 \text{ Байт/с}} \approx 203 \text{ FLOP/байт}.$$

Фактическая плотность (40 FLOP/байт) значительно ниже теоретической, что указывает на то, что реальная работа не является чисто compute-bound. При этом для batch size 8 данные ещё помещаются в кэш, поэтому накладные расходы на доступ к глобальной памяти невелики. Приведенные оценки помогают объяснить резкое увеличение времени и предположить, где возникают узкие места.

## 2.3 Выводы по первому эксперименту

- `torch.compile` стабильно ускоряет инференс UNet на 10–18% для batch size до 16 без потери качества.

- Наблюдается аномальный рост времени при batch size 16, связанный с выходом объёма обрабатываемых данных за пределы кэша L2 GPU и, как следствие, увеличением числа обращений к глобальной памяти.
- Для batch size 2–8 модель работает в режиме, где данные помещаются в кэш, и производительность ограничена главным образом накладными расходами на запуск ядер. Компиляция эффективно сокращает эти накладные расходы.
- Дальнейшие эксперименты с TVM, квантизацией и спарсификацией позволят оценить, можно ли достичь более существенного ускорения за счёт более глубоких оптимизаций, особенно для больших батчей, где узким местом становится работа с памятью.

### 3 План дальнейших работ

По результатам первого этапа сформирован следующий план экспериментов на оставшиеся три недели.

- **Оптимизация с torch.compile в различных режимах** (1-8 апреля). Будут протестированы режимы `max-autotune` и `reduce-overhead` для выявления наилучшей конфигурации компиляции.
- **Альтернативный компилятор TVM** (1-8 апреля). Модель будет экспортирована в ONNX и преобразована в TVM с использованием Relax/TIR для глубокой оптимизации под CUDA.
- **Квантизация** (9-16 апреля). Реализация пост-тренировочной квантизации (PTQ) и, при необходимости, квантизационного обучения (QAT) для int8, оценка влияния на качество и скорость.
- **Спарсификация** (9-16 апреля). Применение unstructured pruning и 2:4 semi-structured sparsity с использованием NVIDIA ASP, fine-tuning для восстановления качества.
- **Профилирование и выявление узких мест** (17-24 апреля). Использование PyTorch Profiler и NVIDIA Nsight Systems для детального анализа bottleneck-ов (память, вычислительные ядра, накладные расходы).
- **Тестирование на другом GPU (H100)** (17-24 апреля). Повтор замеров на более современном GPU для оценки масштабируемости полученных ускорений.
- **Составление отчета** (23-25 апреля). Получение финальных результатов, их сравнение и анализ. Окончательно оформление репозитория проекта, подготовка отчета и составление презентации с полученными результатами для дальнейшей защиты.

## Список литературы

- [1] Carvana Image Masking Challenge, Kaggle. <https://www.kaggle.com/c/carvana-image-masking-challenge>
- [2] Ronneberger, O., Fischer, P., Brox, T. (2015). U-Net: Convolutional Networks for Biomedical Image Segmentation. *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*.
- [3] Milesial. (2020). Pytorch-UNet. <https://github.com/milesial/Pytorch-UNet>